

UBER:

1. Arquitetura do Sistema:

Evolução: Do Monólito aos microsserviços.

O Uber começou sua trajetória com uma arquitetura monolítica. Um banco de dados MySQL armazena todos os dados, e todas as funções (gestão de passageiros, cobrança, notificações, pagamentos e gestão de corridas). Com o crescimento global, essa estrutura tornou-se insustentável. Em 2014, o Uber adotou a arquitetura de microsserviços para garantir melhor velocidade e desempenho. Com o tempo, evoluiu ainda mais para uma abordagem chamada DOMA (Domain-Oriented Microservices Architecture). Atualmente, os 4.500 microsserviços stateless do Uber são implantados mais de 100.000 vezes por semana por 4.000 engenheiros e sistemas autônomos. Em 2023, o Uber selecionou a Oracle Cloud Infrastructure (OCI) para transição de sua infraestrutura principal, migrando milhares de microsserviços, múltiplas plataformas de armazenamento de dados e dezenas de modelos de IA para a nuvem. Oracle.

2. Mecanismos de Comunicação:

O Uber construiu sua própria malha de serviços, conhecida como Uber Gateway (baseada em NGINX), para gerenciar a comunicação entre serviços. Esse sistema utiliza protocolos de alto desempenho como gRPC e QUIC, além de Apache Thrift para definição de API e serialização, e circuit breakers para tolerância a falhas.

3. Bancos de Dados:

A arquitetura de dados do Uber utiliza diferentes tecnologias para diferentes casos de uso: Docstore (banco proprietário sobre MySQL): para armazenamento de dados de longo prazo. Riak e Cassandra: para acesso de baixa latência. Redis: para cache e filas para melhorar a performance. Apache Pinot, Kafka e Flink: para processamento de dados em tempo real.

4. Desempenho e Tolerância a Falhas:

Circuit breakers: para prevenir falhas em cascata.

Redundância: réplicas ativas-ativas implantadas em múltiplas regiões e mecanismo de fallback.

NETFLIX:

1. Arquitetura do Sistema:

Em 2008, uma grande falha no banco de dados Oracle monolítico da Netflix causou uma interrupção de vários dias, levando a uma reformulação arquitetural completa. Ao longo de oito anos, a Netflix migrou para um sistema baseado em microsserviços, permitindo escalabilidade horizontal e melhor confiabilidade.

A Netflix opera com uma separação clara entre dois planos:

Plano de Controle (AWS): todas as interações do usuário antes da reprodução, navegação, recomendações e gestão de conta são gerenciadas por microsserviços rodando na Amazon Web Services.

Plano de Dados (Open Connect): quando um título é selecionado, a CDN proprietária da Netflix assume para fazer o streaming do vídeo com eficiência.

2. Mecanismos de Comunicação:

A Netflix utiliza gRPC e REST APIs para comunicação de alto desempenho entre microsserviços. No front-end, emprega GraphQL e Falcor para busca eficiente de dados e otimização de respostas de API. Substack.

3. Bancos de Dados:

A Netflix utiliza uma combinação de tecnologias de banco de dados:

Apache Cassandra e DynamoDB: NoSQL para preferências de usuários, histórico de visualização e metadados.

MySQL: para dados transacionais como faturamento e pagamentos.

Amazon S3 e EBS: para logs, metadados e assets de vídeo.

Apache Kafka e AWS Kinesis: para streaming de eventos, analytics em tempo real e processamento de logs.

4. Desempenho e Tolerância a Falhas:

A Netflix inventou a engenharia do caos em 2011 com o Chaos Monkey, que encerra aleatoriamente instâncias de VM em produção para garantir que os serviços continuem funcionando mesmo diante de falhas não planejadas. O Chaos Gorilla simula a indisponibilidade de uma zona de disponibilidade inteira da Amazon. Além disso, o Latency Monkey induz atrasos nas comunicações cliente-servidor para testar como o sistema se comporta sob condições degradadas. O Netflix trata a falha como algo inevitável, portanto projeta seus sistemas para ela. O sistema incorpora técnicas como bulkheading, retries e circuit breakers, além de ferramentas de orquestração automatizada como o Netflix Conductor para gerenciar fluxos de trabalho de microsserviços em escala.